Innominatum CONSULTORIA

Ciclo de Vida do Software

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parceiros para o Crescimento
PORTUGAL DIGITAL

Objetivo

Compreender o ciclo de vida de uma aplicação Angular, desde o código-fonte até ao artefacto distribuível, dominando ferramentas de *build*, automação com npm scripts, gestão de variáveis de ambiente e o conceito de pipeline como orquestrador de tarefas.

Objetivos Específicos

Ao final da sessão, os formandos deverão ser capazes de:

- Descrever o ciclo de vida do software, desde o código-fonte ao artefacto distribuível;
- Explicar o que acontece internamente durante um ng build;
- Configurar npm scripts como orquestrador de tarefas;
- Gerir variáveis de ambiente para contextos dev e prod;
- Explicar o conceito de pipeline como forma de encadear tarefas automaticamente.

Innominatum CONSULTORIA

Ciclo de Vida do Software

Do código-fonte ao artefacto distribuível

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parceiros para o Crescimento
PORTUGAL DIGITAL

Do Código ao Artefacto

Num projeto Angular, o código que escrevemos não é o que o browser executa. Existe um processo intermediário que converte, compacta e otimiza tudo antes da distribuição.

- **Fase 1**: Desenvolvimento – O programador escreve TypeScript, HTML e SCSS. O Angular CLI oferece um servidor local (ng serve) com hot reload.
- **Fase 2**: Build – O ng build compila e gera ficheiros estáticos prontos para distribuição na pasta dist/.
- **Fase 3**: Distribuição – O artefacto em dist/ é copiado para um servidor web (ex: nginx, Vercel, Netlify) que o serve aos utilizadores.

O que acontece num ng build

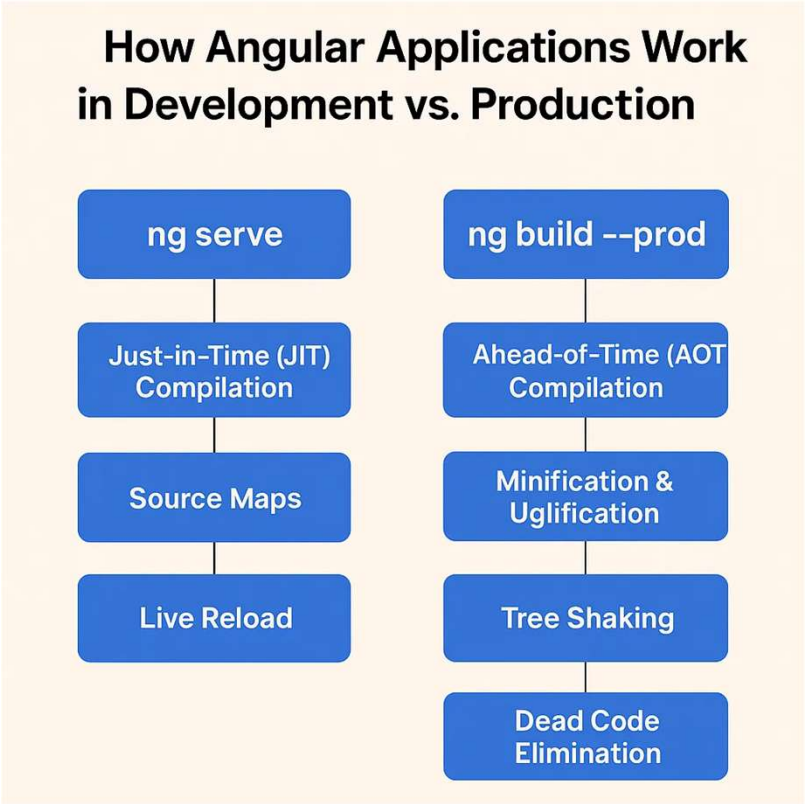
O comando *ng build* processa o código Angular através de várias etapas automáticas antes de gerar os ficheiros finais.

Transpilação TypeScript → Bundling → Minificação → Tree-Sharking

O que acontece num ng build

- **Transpilação TypeScript**: o TypeScript não corre no browser. O compilador tsc converte cada ficheiro .ts para JavaScript compatível.
- **Bundling**: os módulos espalhados por vários ficheiros são agrupados em *bundles* (main.js, polyfills.js, styles.css) pelo Webpack ou esbuild.
- **Minificação**: espaços, comentários e nomes longos são removidos/abreviados para reduzir o tamanho dos ficheiros.
- **Tree-shaking**: código que não é usado em nenhum ponto da aplicação é eliminado automaticamente.

O que acontece num ng build



Innominatum CONSULTORIA

npm scripts

O orquestrador de tarefas no package.json

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parceiros para o Crescimento
PORTUGAL DIGITAL

npm scripts no package.json

A secção “scripts” do package.json permite definir atalhos para comandos executados com:

```
npm run [nome-do-script]
```

npm scripts no package.json

Cada script é uma cadeia de texto que o npm passa ao shell do sistema operativo, pelo que podem ser compostos com && (sequencial) ou || (alternativa).

Exemplo: "ci": "npm run lint && npm run build:prod"

Garante que o lint é validado antes de iniciar o build de produção. Se o lint falhar, o build não arranca.

npm scripts no package.json

Scripts com prefixo pre/post são executados automaticamente antes/depois.

Exemplo: "prebuild" corre antes de "build" sem precisar de ser invocado explicitamente.

Os Scripts Essenciais

- “start”: “ng serve” – servidor de desenvolvimento com *hot reload;*
- “build”: “ng build” – build de desenvolvimento (com source maps);
- “build:prod”: “ng build --configuration production” – build otimizado para produção;
- “lint”: “ng lint” – valida o código com as regras ESLint configuradas;
- “test”: “ng test” – executa os testes unitários com Karma/Jasmine;
- “ci”: “npm run lint && npm run build:prod” – pipeline local completo.

Os Scripts Essenciais

```
{
  "name": "sinapse",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "private": true,
  "scripts": {
    "install:all": "npm --prefix backend install && npm --prefix
frontend install",
    "dev": "concurrently \"npm --prefix backend run dev\" \"npm
--prefix frontend run dev\"",
    "build": "npm --prefix frontend run build",
    "start": "npm --prefix backend run start"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "devDependencies": {
    "concurrently": "^9.2.1"
  },
  "dependencies": {
    "mongodb": "^7.0.0"
  }
}
```

Os Scripts Essenciais

```
"scripts": {
  "install:all": "npm --prefix backend install && npm --prefix
    frontend install",
  "dev": "concurrently \"npm --prefix backend run dev\" \"npm
    --prefix frontend run dev\"",
  "build": "npm --prefix frontend run build",
  "start": "npm --prefix backend run start"
},
```

Innominatum CONSULTORIA

Variáveis de Ambiente

environment.ts, .env e a gestão de contextos

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parceiros para o Crescimento
PORTUGAL DIGITAL

Variáveis de Ambiente no Angular

O Angular usa ficheiros **environment.ts para gerir configurações** que variam entre contextos (**dev**, **staging**, **production**).

- src/environments/environment.ts: valores para **desenvolvimento local**;
- src/environments/environment.prod.ts: valores para **produção**.

Variáveis de Ambiente no Angular

```
export const environment = {
  production: false,
  apiKey: 'd4f12f61541640bcc35e09bbd454ea5f',
  baseUrl: 'http://api.themoviedb.org/3',
  images: 'http://image.tmdb.org/t/p'
};
```

Variáveis de Ambiente no Angular

```
import { environment } from 'src/environments/environment';
```

Variáveis de Ambiente no Angular

O Angular CLI **substitui automaticamente o ficheiro certo** no momento do build, conforme o flag --configuration. O código importa sempre environment.ts.

Para **segredos sensíveis** (tokens de API, credenciais), usar ficheiros .env que são carregados no ambiente CI/CD e nunca devem ser submetidos/adicionados (commit) ao repositório.

dev vs prod: o que muda

O Angular suporta dois modos de *build* com características distintas, controlados pelo *flag* --configuration.

- Modo development (ng build);
- Modo production (ng build --configuration production).

dev vs prod: o que muda

- **Modo development (ng build)**: sem minificação, com source maps detalhados, otimizações desativadas. O bundle é maior mas mais fácil de depurar.
- **Modo production (ng build --configuration production)**: minificação ativa, tree-shaking agressivo, AOT (*Ahead-of-Time*) compilation. Bundle muito mais pequeno e rápido.

dev vs prod: o que muda

Na prática usa-se *development* no dia a dia e *production* para deploy.

Nunca fazer deploy com o modo development ativado.

Innominatum CONSULTORIA

Artefactos de Build

O que é o dist/ e para que serve

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parceiros para o Crescimento
PORTUGAL DIGITAL

O que está no dist/

Após um *ng build*, a pasta dist/ contém tudo o necessário para servir a aplicação. É um artefacto autónomo que não depende do Node.js em produção, composto essencialmente por:

- **main.js**: código da aplicação compilado.
- **polyfills.js**: compatibilidade com browsers.
- **styles.css**: estilos globais.
- **Lazy chunks**: módulos carregados on-demand.
- **Assets**: imagens, fontes e ficheiros copiados de src/assets.
- **index.html**: ponto de entrada da aplicação.

Este artefacto pode ser copiado diretamente para qualquer servidor HTTP estático (nginx, Vercel, S3) sem qualquer compilação adicional.

Innominatum CONSULTORIA

Versionamento Automático

npm version, SemVer e changelog

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parceiros para o Crescimento
PORTUGAL DIGITAL

npm version e SemVer

Relembrando:

O versionamento semântico (SemVer) define um formato padronizado:

MAJOR.MINOR.PATCH (ex: 2.4.1).

- PATCH (2.4.1 → 2.4.2): correção de bugs sem quebrar compatibilidade;
- MINOR (2.4.1 → 2.5.0): nova funcionalidade retrocompatível;
- MAJOR (2.4.1 → 3.0.0): quebra de compatibilidade.

npm version e SemVer

O comando *npm version patch/minor/major* atualiza o package.json, cria um *commit* e uma tag Git automaticamente.

- Changelog automático: ferramentas como standard-version ou conventional-changelog geram o CHANGELOG.md a partir dos Conventional Commits.

Innominatum CONSULTORIA

O Conceito de Pipeline

Encadear tarefas automaticamente

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parceiros para o Crescimento
PORTUGAL DIGITAL

O Pipeline como Orquestrador

Um pipeline é uma sequência de tarefas executadas automaticamente e em ordem. Cada passo só começa se o anterior terminou com sucesso.

Este conceito é a base do CI/CD (Integração e Entrega Contínua) que veremos com GitHub Actions nas sessões seguintes.

O Pipeline como Orquestrador

Exemplo com npm scripts:

lint → test → build:prod

- Se o lint falhar, o build não acontece; se o build falhar, o deploy não acontece.
- A automação elimina erros humanos e garante consistência. Ninguém faz *deploy* sem ter passado pelos testes e pelo lint.

Aula Prática

eslint.config.js > ...

```
1  // @ts-check
2  const { defineConfig } = require('eslint/config');
3  const tseslint = require('typescript-eslint');
4  const angular = require('angular-eslint');
5
6  module.exports = defineConfig([
7    tseslint.configs.base,
8    {
9      files: ['**/*.ts'],
10     processor: angular.processInlineTemplates,
11     rules: {
12       '@typescript-eslint/no-unused-vars': 'error',
13     },
14   }
15 ]);
```

Aula Prática

{} package.json > { } scripts > abc watch

▷Debug

```
4   "scripts": {
5     "ng": "ng",
6     "start": "ng serve",
7     "build": "ng build",
8     "watch": "ng build --watch --configuration development",
9     "test": "ng test",
10    "lint": "ng lint",
11    "ci": "ng lint || ng build"
12  },
```

Innominatum CONSULTORIA

Ciclo de Vida do Software

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parceiros para o Crescimento
PORTUGAL DIGITAL